Release 1 : Prise de Rendez-vous

Authentification & Gestion des Rendez-vous

MediConnect — Plateforme e-Santé

NestJS + Angular 17 • PFE 2026

2025 – 2026

3.1 Introduction

Ce chapitre se concentre sur les étapes de réalisation de notre deuxième release en mettant l'accent sur la création et le raffinement du backlog sprint 0 et sprint 1, la conception et la réalisation.

Le sprint 0 couvre les fonctionnalités d'authentification et d'inscription pour l'ensemble des acteurs de la plateforme e-Santé (patients, médecins, pharmaciens et administrateur). Le sprint 1 se concentre sur la prise de rendez-vous médicaux, incluant la recherche de médecins, la sélection de créneaux en temps réel, les confirmations et rappels automatiques.

3.2 Backlog de Release 1

Pour planifier les tâches à accomplir, nous allons créer le backlog du release 2 présenté sur le tableau 3.1. L'estimation est utilisée pour évaluer la charge de travail associée à chaque tâche.

User Story (Backlog Produit)	Priorité	Évaluation	Sprint	Acteur	Workflow
Sprint 0 — *Authentification & Inscription*					
En tant qu'administrateur, je veux m'authentifier pour accéder au back-office	1	Moyen	Sprint 0	Admin	Auth
En tant que patient, je veux m'inscrire (nom, email, mot de passe, profil médical de base)	1	Moyen	Sprint 0	Patient	Auth
En tant que patient, je veux m'authentifier	1	Moyen	Sprint 0	Patient	Auth
En tant que patient, je veux réinitialiser mon mot de passe par email	1	Faible	Sprint 0	Patient	Auth
En tant que médecin, je veux m'inscrire (RPPS, spécialité, cabinet)	1	Moyen	Sprint 0	Médecin	Auth
En tant que médecin, je veux m'authentifier	1	Moyen	Sprint 0	Médecin	Auth
En tant que médecin, je veux réinitialiser mon mot de passe	1	Faible	Sprint 0	Médecin	Auth
En tant que pharmacien, je veux m'inscrire (numéro ordre, officine)	1	Moyen	Sprint 0	Pharmacien	Auth
En tant que pharmacien, je veux m'authentifier	1	Moyen	Sprint 0	Pharmacien	Auth
Sprint 1 — *W1 — Prise de Rendez-vous*					
En tant que patient, je veux rechercher un médecin par spécialité, localisation et disponibilité	2	Fort	Sprint 1	Patient	W1 RDV
En tant que patient, je veux consulter le profil d'un médecin (photo, avis, tarifs)	2	Moyen	Sprint 1	Patient	W1 RDV
En tant que patient, je veux sélectionner un créneau disponible en temps réel (physique ou téléconsultation)	2	Fort	Sprint 1	Patient	W1 RDV
En tant que médecin, je veux recevoir une notification instantanée lors d'une demande de RDV	2	Fort	Sprint 1	Médecin	W1 RDV

User Story (Backlog Produit)	Priorité	Évaluation	Sprint	Acteur	Workflow
En tant que médecin, je veux accepter ou refuser une demande de RDV en 1 clic (statut VALIDÉ / REFUSÉ)	2	Fort	Sprint 1	Médecin	W1 RDV
En tant que patient, je veux recevoir une confirmation de RDV par email et SMS	2	Moyen	Sprint 1	Patient	W1 RDV
En tant que patient, je veux recevoir un rappel automatique 24h avant le RDV avec le lien de téléconsultation	2	Moyen	Sprint 1	Patient	W1 RDV
En tant que patient, je veux annuler ou reporter un RDV depuis mon espace	2	Moyen	Sprint 1	Patient	W1 RDV

Tableau 3.1 – Backlog de Release 2

3.3 Raffinement de Release 1

Dans cette section nous examinons ces cas d'utilisation :

- S'inscrire
- S'authentifier
- Prendre un rendez-vous
- Gérer les rendez-vous

3.3.1 Raffinement de Sprint 0

Raffinement du cas "S'inscrire"

La figure 3.1 offre une représentation visuelle du cas "S'inscrire", décrivant le processus que les utilisateurs suivent pour s'enregistrer sur la plateforme e-Santé :

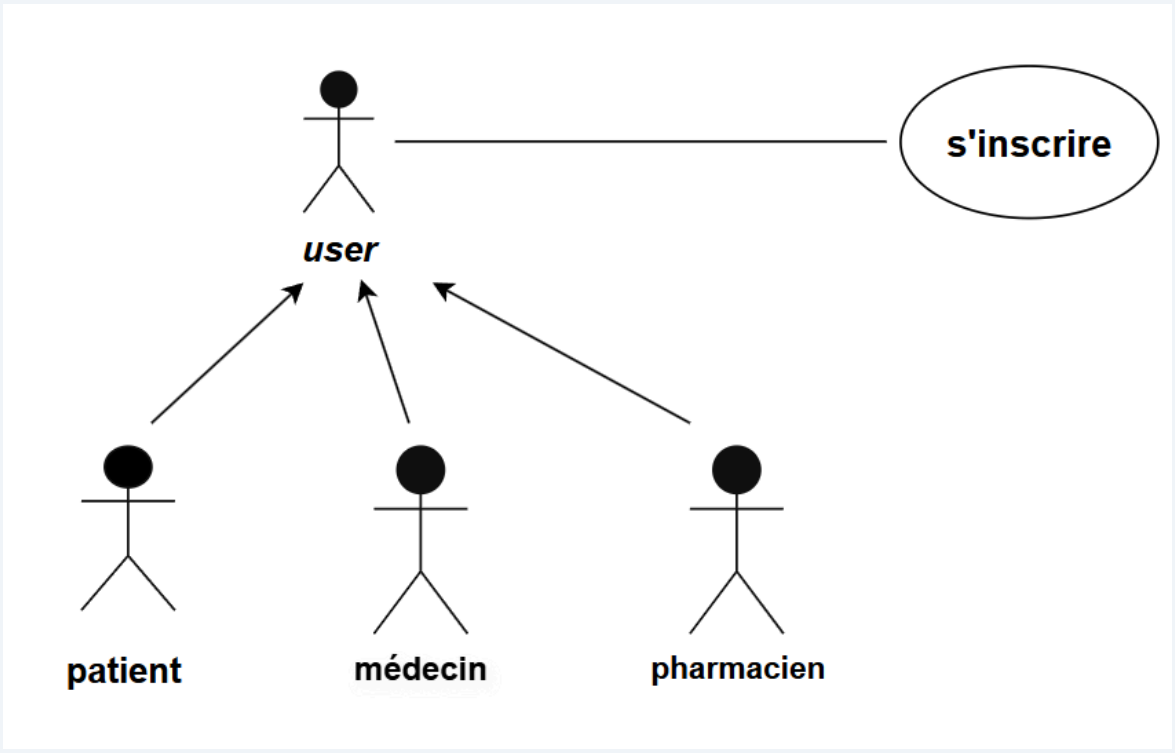


Figure 3.1 – Décomposition de cas d'utilisation « S'inscrire »

Dans cette figure 3.2 on trouve le diagramme de séquence associé au cas "S'inscrire" :

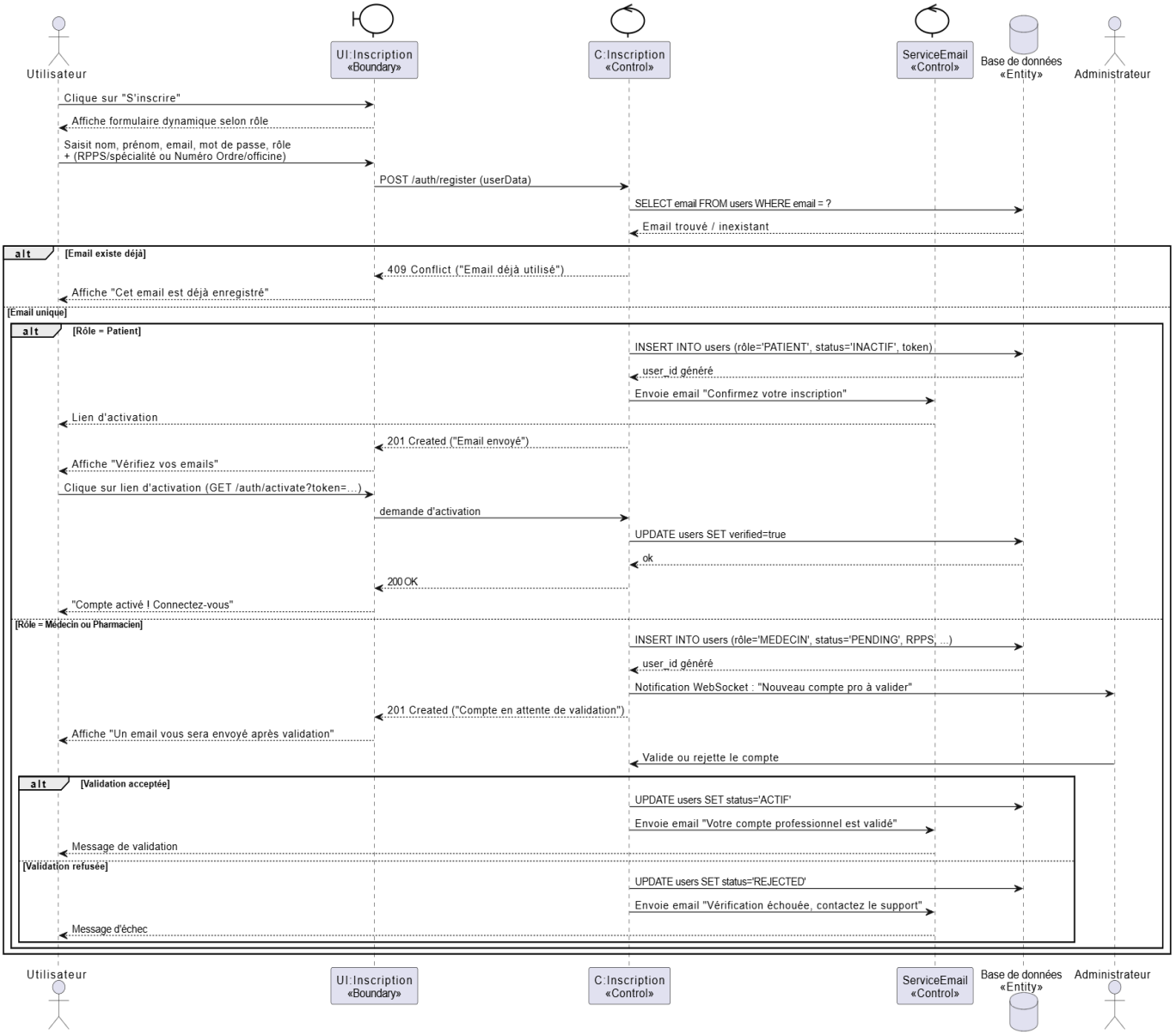


Figure 3.2 –Diagramme de séquence Analytique du cas « S'inscrire »

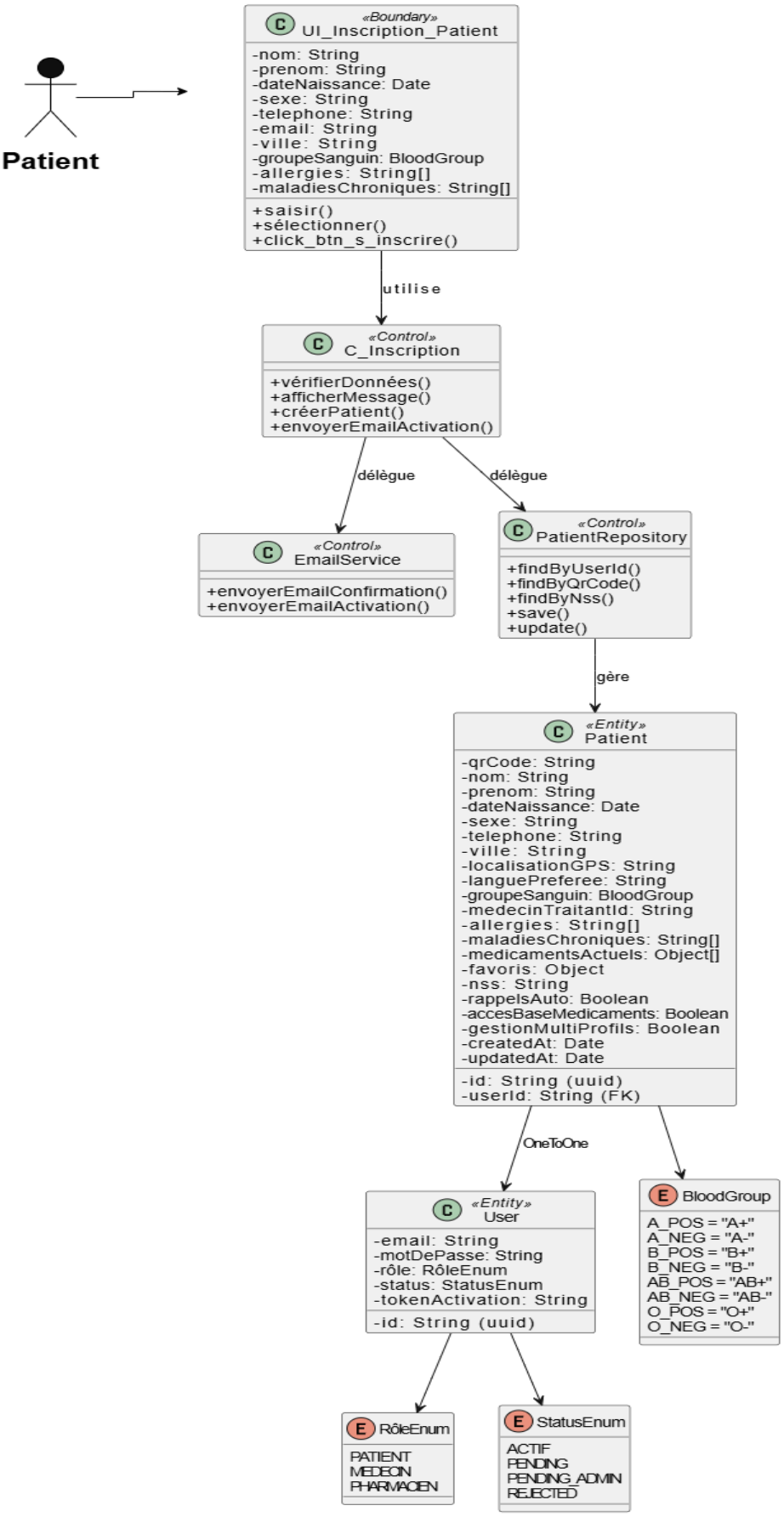


Figure 3.3 –Diagramme de classe Analytique du cas « S'inscrire patient »

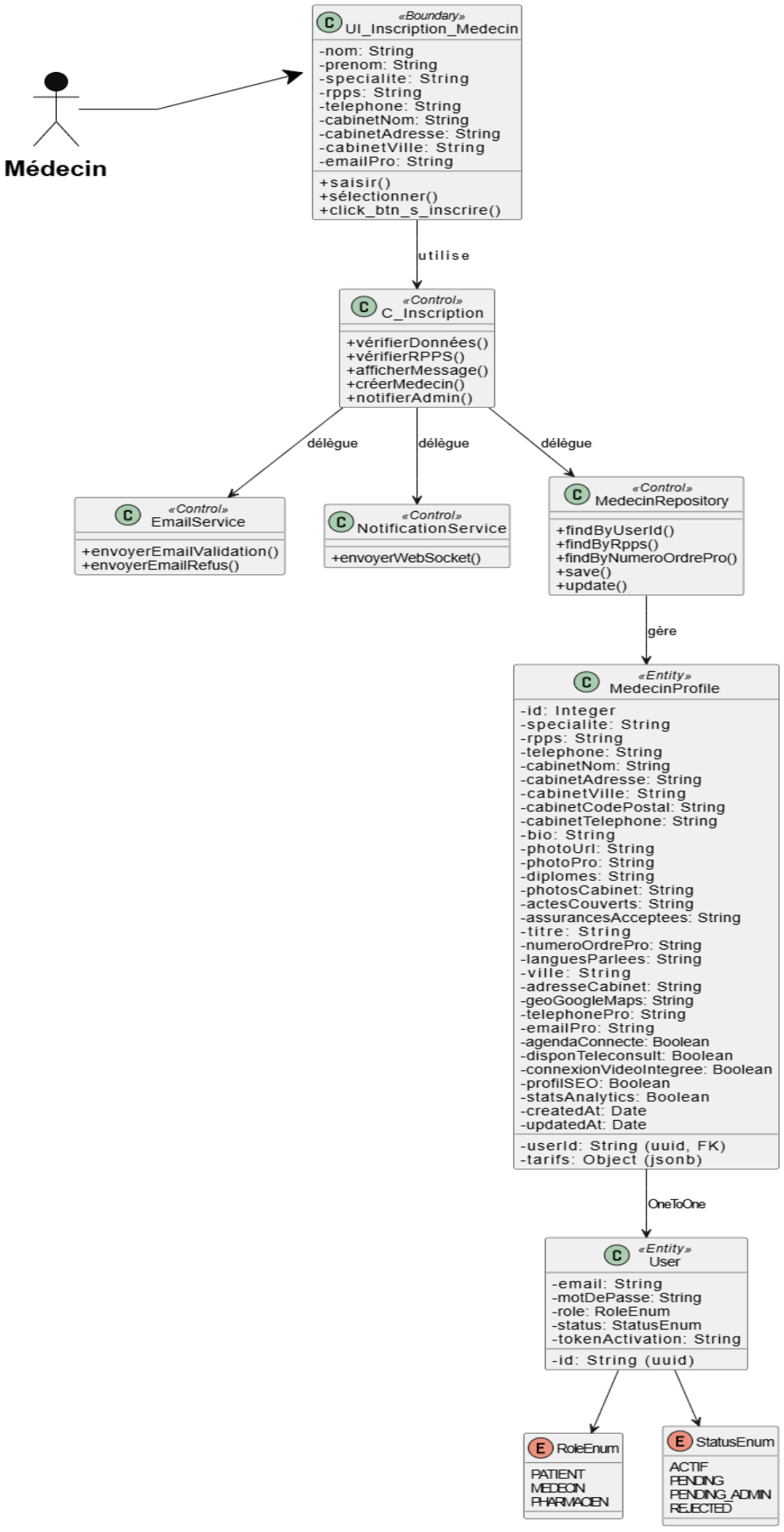

Médecin
«Boundary»
UI_Inscription_Medecin
-nom: String
-prenom: String
-specialite: String
-rpps: String
-telephone: String
-cabinetNom: String
-cabinetAdresse: String
-cabinetVille: String
-emailPro: String
+saisir()
+sélectionner()
+click_btn_s_inscrire()
utilise
«Control»
C_Inscription
+vérifierDonnées()
+vérifierRPPS()
+afficherMessage()
+créerMedecin()
+notifierAdmin()
délègue
délègue
délègue
«Control»
EmailService
+envoyerEmailValidation()
+envoyerEmailRefus()
«Control»
NotificationService
+envoyerWebSocket()
«Control»
MedecinRepository
+findByUserId()
+findByRpps()
+findByNumeroOrdrePro()
+save()
+update()
gère
«Entity»
MedecinProfile
-id: Integer
-specialite: String
-rpps: String
-telephone: String
-cabinetNom: String
-cabinetAdresse: String
-cabinetVille: String
-cabinetCodePostal: String
-cabinetTelephone: String
-bio: String
-photoUrl: String
-photoPro: String
-diplomes: String
-photosCabinet: String
-actesCouverts: String
-assurancesAcceptees: String
-titre: String
-numeroOrdrePro: String
-languesParlees: String
-ville: String
-adresseCabinet: String
-geoGoogleMaps: String
-telephonePro: String
-emailPro: String
-agendaConnecte: Boolean
-disponTeleconsult: Boolean
-connexionVideoIntegree: Boolean
-profilSEO: Boolean
-statsAnalytics: Boolean
-createdAt: Date
-updatedAt: Date
-userId: String (uuid, FK)
-tarifs: Object (jsonb)
OneToOne
«Entity»
User
-email: String
-motDePasse: String
-role: RoleEnum
-status: StatusEnum
-tokenActivation: String
-id: String (uuid)
RoleEnum
PATIENT
MEDECIN
PHARMACIEN
StatusEnum
ACTIF
PENDING
PENDING_ADMIN
REJECTED

Figure 3.4 –Diagramme de classe Analytique du cas « S'inscrire médecin»

Le tableau 3.2 illustre en détail le processus de décomposition du cas "S'inscrire" :

Cas	S'inscrire
Acteur	-Patient -Médecin -Pharmacien
Condition préalable	La plateforme est opérationnelle. Adresse mail et mot de passe valides.
Condition résultante	Inscription réussie.
Description de scénario	- La plateforme affiche une page d'inscription. - L'utilisateur remplit ses informations personnelles. - L'utilisateur appuie sur le bouton « S'inscrire ». - La plateforme inspecte les informations. - La plateforme indique un avis de succès.
Scénarii	- La plateforme indique une imprécision si les informations sont inexactes.

Tableau 3.4 – Décomposition du cas « S'inscrire »

Raffinement du cas "S'authentifier"

La figure 3.2 démontre une décomposition de cas d'utilisation "S'authentifier" :

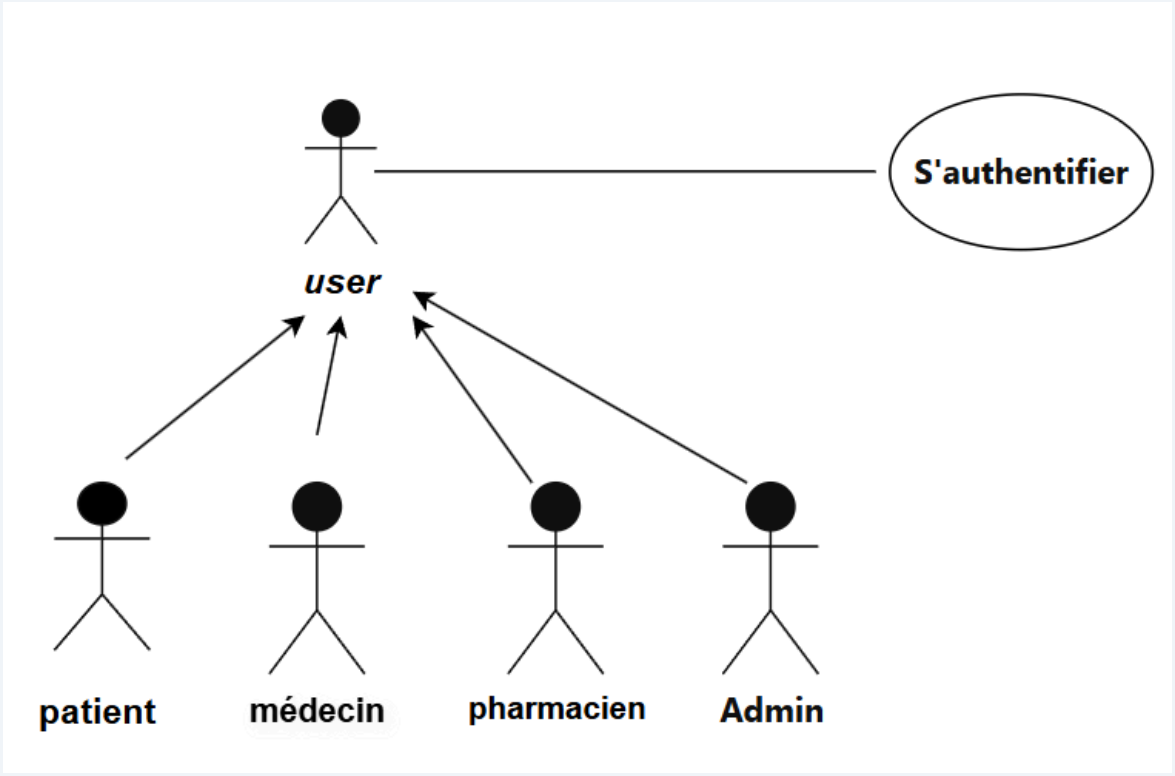


Figure 3.2 – Raffinement du cas d'utilisation « S'authentifier »

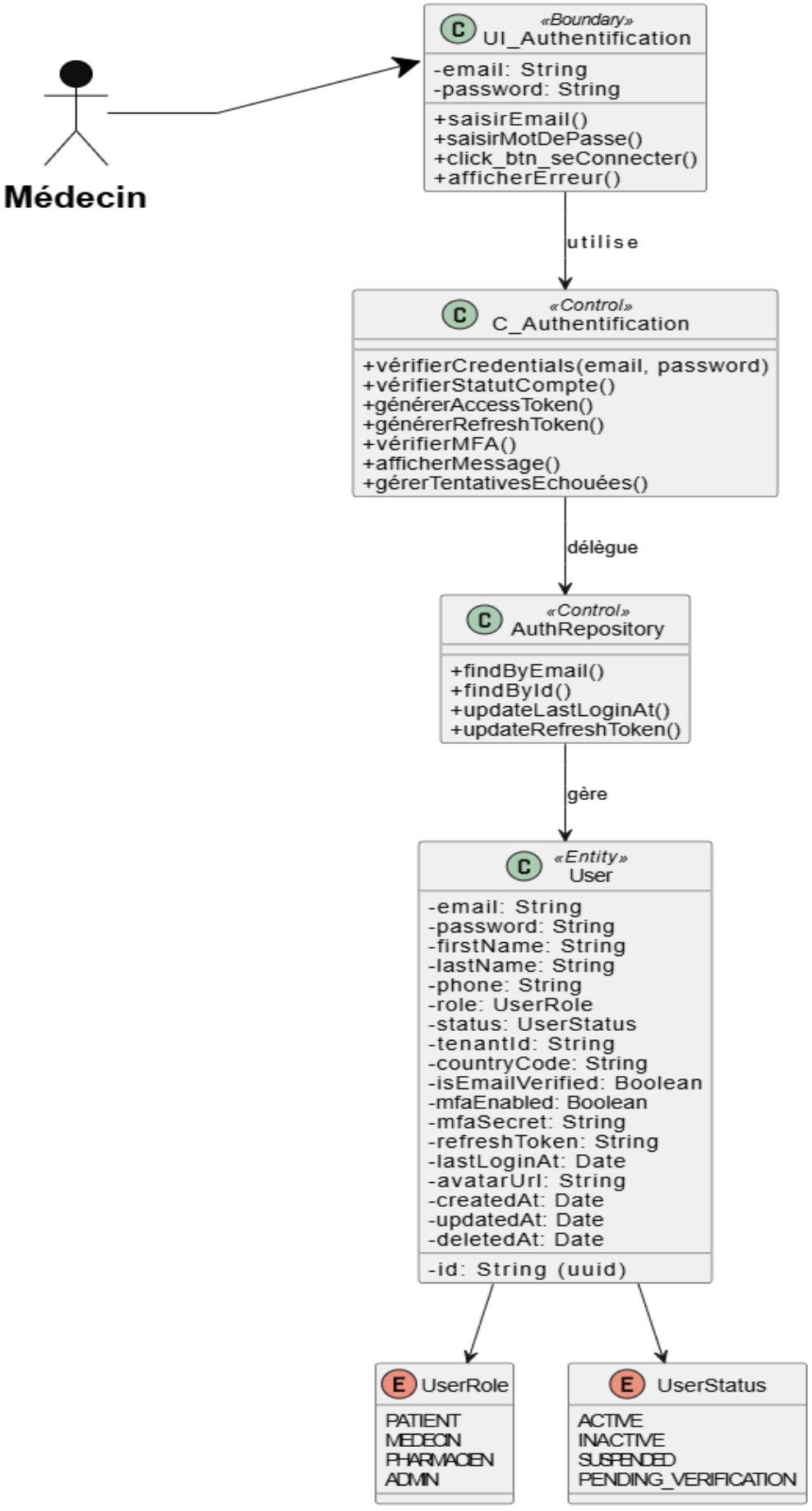
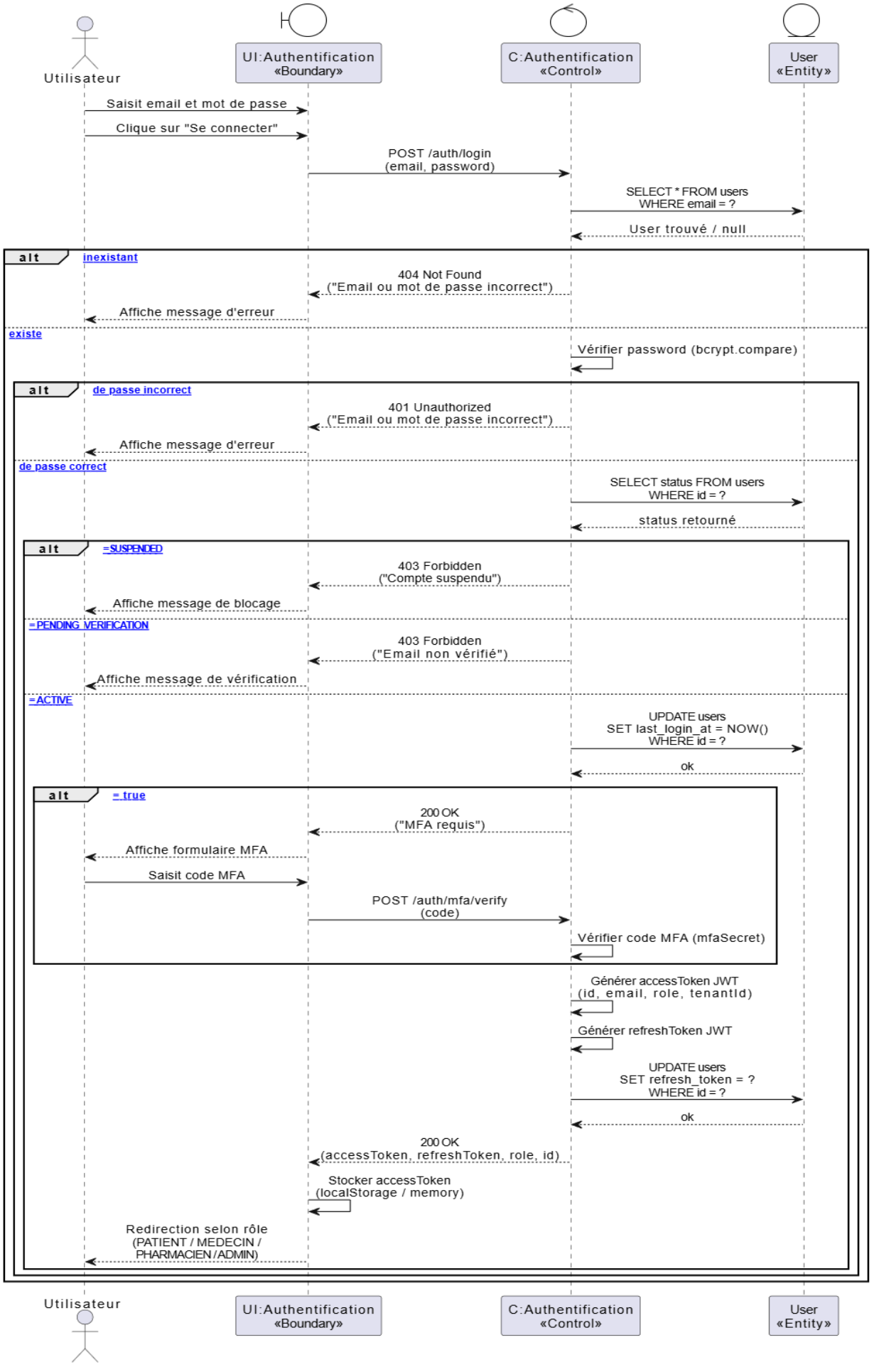


Figure 3.2 –Diagramme de séquence Analytique du cas « S'authentifier »


Utilisateur
UI:Authentification «Boundary»
C:Authentification «Control»
User «Entity»
Saisit email et mot de passe
Clique sur "Se connecter"
POST /auth/login (email, password)
SELECT * FROM users WHERE email = ?
User trouvé / null
alt
inexistant
404 Not Found ("Email ou mot de passe incorrect")
Affiche message d'erreur
existe
Vérifier password (bcrypt.compare)
alt
de passe incorrect
401 Unauthorized ("Email ou mot de passe incorrect")
Affiche message d'erreur
de passe correct
SELECT status FROM users WHERE id = ?
status retourné
alt
=SUSPENDED
403 Forbidden ("Compte suspendu")
Affiche message de blocage
=PENDING_VERIFICATION
403 Forbidden ("Email non vérifié")
Affiche message de vérification
=ACTIVE
UPDATE users SET last_login_at = NOW() WHERE id = ?
ok
alt
= true
200 OK ("MFA requis")
Affiche formulaire MFA
Saisit code MFA
POST /auth/mfa/verify (code)
Vérifier code MFA (mfaSecret)
Générer accessToken JWT (id, email, role, tenantId)
Générer refreshToken JWT
UPDATE users SET refresh_token = ? WHERE id = ?
ok
200 OK (accessToken, refreshToken, role, id)
Stocker accessToken (localStorage / memory)
Redirection selon rôle (PATIENT / MEDECIN / PHARMACIEN / ADMIN)
Utilisateur
UI:Authentification «Boundary»
C:Authentification «Control»
User «Entity»

Le Tableau 3.3 illustre la décomposition détaillée du cas d'utilisation "S'authentifier" :

Cas	S'authentifier
Acteur	-Administrateur -Médecin -Patient -Pharmacien
Condition préalable	La plateforme est opérationnelle.
Condition résultante	Utilisateur authentifié.
Description de scénario	- La plateforme affiche la page d'authentification. - L'utilisateur saisit son adresse e-mail et son mot de passe. - La plateforme inspecte l'adresse e-mail et le mot de passe. - La plateforme oriente vers la page suivante.
Scénarii	- La plateforme indique une imprécision si les informations sont inexactes.

Tableau 3.3 – Raffinement du cas « S'authentifier »

3.3.2 Raffinement de Sprint 1

Raffinement du cas "Prendre un rendez-vous"

La figure 3.3 présente la décomposition détaillée du cas "Prendre un rendez-vous" :

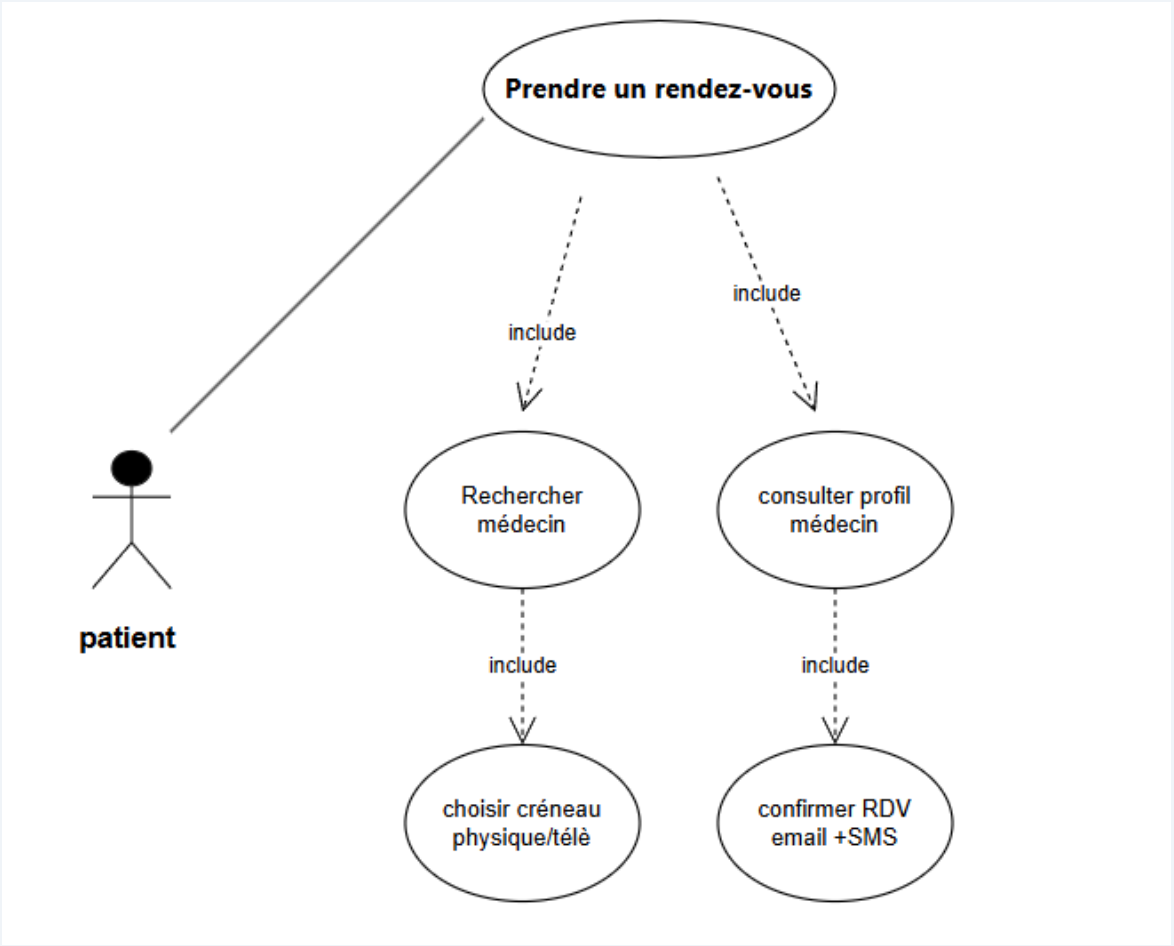


Figure 3.3 – Décomposition de cas d'utilisation « Prendre un rendez-vous »

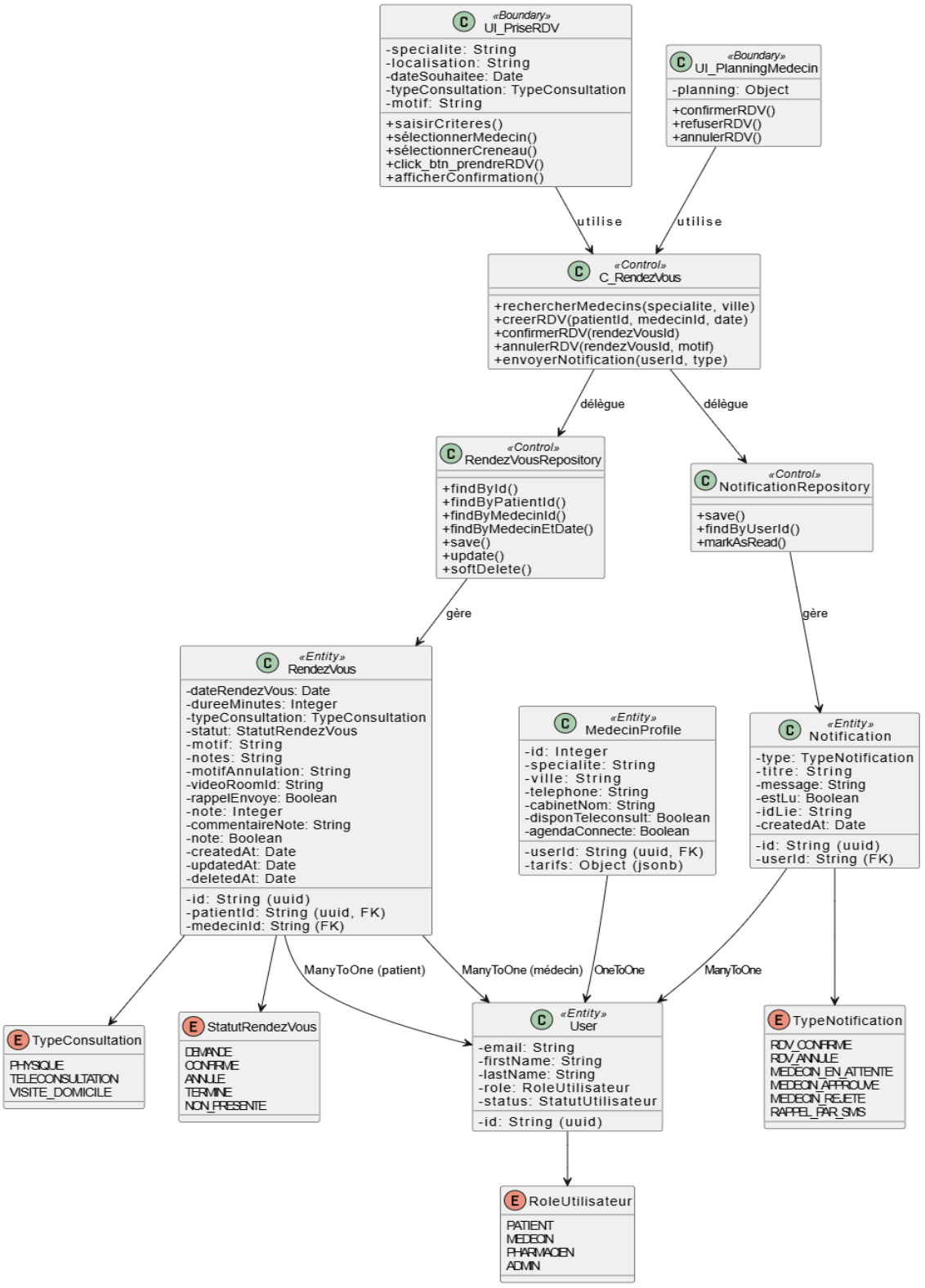
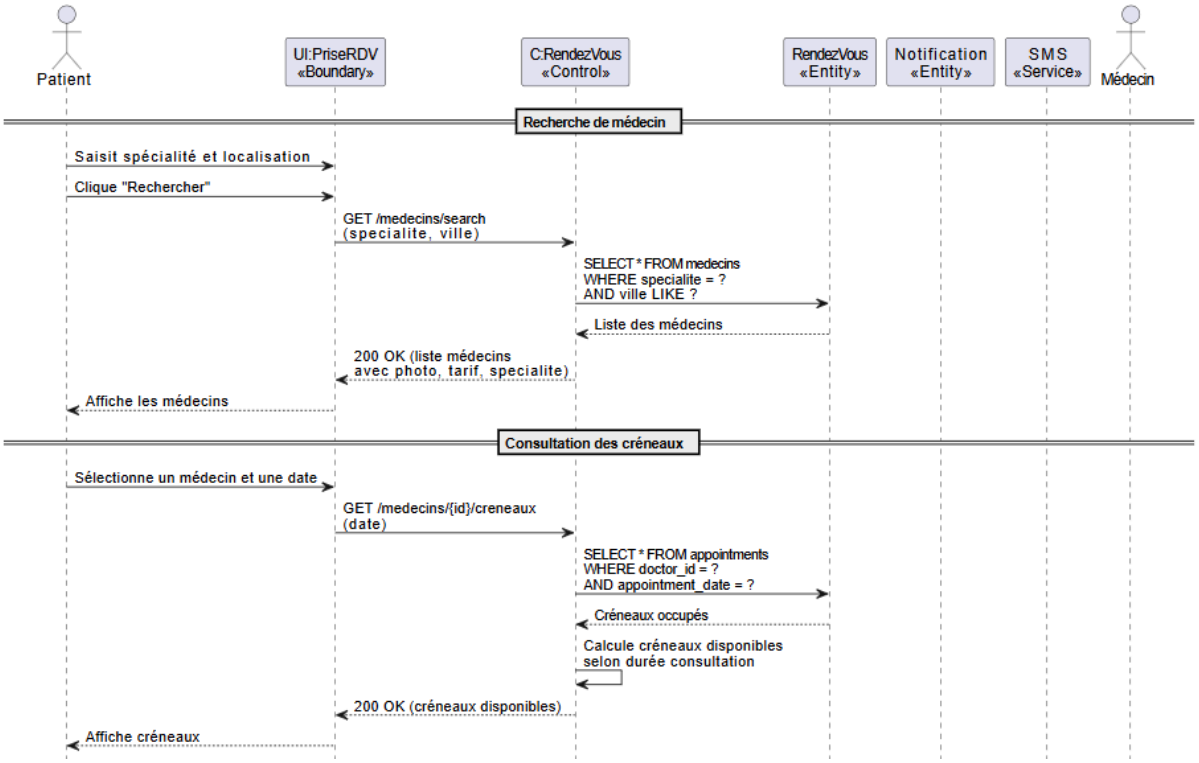


Figure 3.3 –Diagramme de classe Analytique du cas « Prendre un rendez-vous »


Patient
UI:PriseRDV «Boundary»
C:RendezVous «Control»
RendezVous «Entity»
Notification «Entity»
SMS «Service»
Médecin
Recherche de médecin
Saisit spécialité et localisation
Clique "Rechercher"
GET /medecins/search (specialite, ville)
SELECT * FROM medecins WHERE specialite = ? AND ville LIKE ?
Liste des médecins
200 OK (liste médecins avec photo, tarif, specialite)
Affiche les médecins
Consultation des créneaux
Sélectionne un médecin et une date
GET /medecins/{id}/creneaux (date)
SELECT * FROM appointments WHERE doctor_id = ? AND appointment_date = ?
Créneaux occupés
Calcule créneaux disponibles selon durée consultation
200 OK (créneaux disponibles)
Affiche créneaux

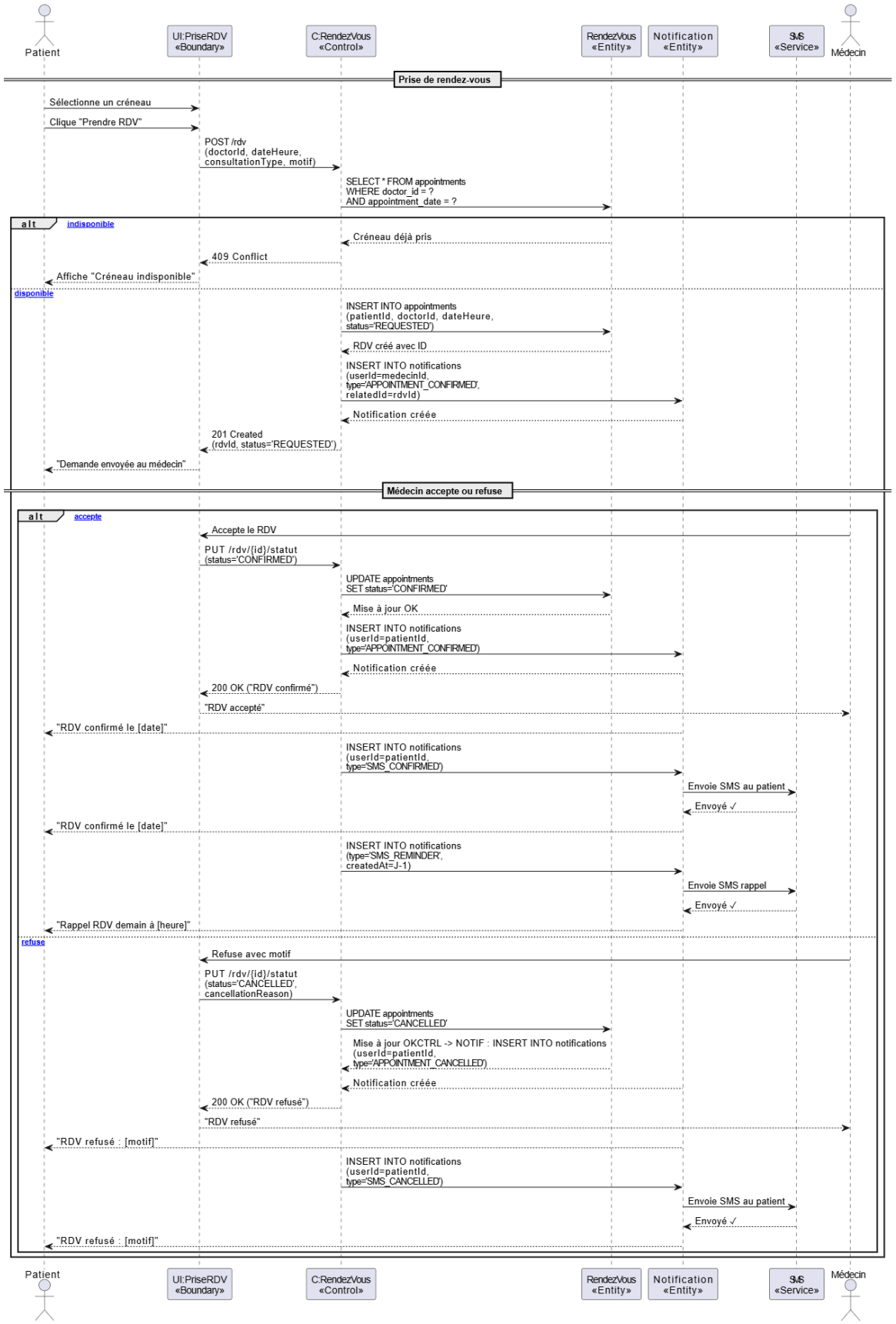


Diagramme de séquence Analytique du cas « Prendre un rendez-vous »

Le Tableau 3.4 présente la décomposition détaillée du cas "Prendre un rendez-vous" :

Cas	Prendre un rendez-vous
Acteur	-Patient
Condition préalable	L'utilisateur est authentifié et la plateforme est opérationnelle.
Condition résultante	Rendez-vous créé et confirmé.
Description de scénario	- Le patient recherche un médecin par spécialité, localisation et disponibilité. - Le patient consulte le profil du médecin (photo, avis, tarifs). - Le patient sélectionne un créneau disponible en temps réel (physique ou téléconsultation). - Le patient confirme la prise de rendez-vous. - La plateforme envoie une confirmation par email et SMS. - Un rappel automatique est envoyé 24h avant le RDV avec le lien de téléconsultation.
Scénarii	- Aucun créneau disponible : la plateforme affiche un message d'indisponibilité.

Tableau 3.4 – Raffinement du cas « Prendre un rendez-vous »

Raffinement du cas "Gérer les rendez-vous"

La figure 3.4 présente le raffinement du cas d'utilisation "Gérer les rendez-vous" :

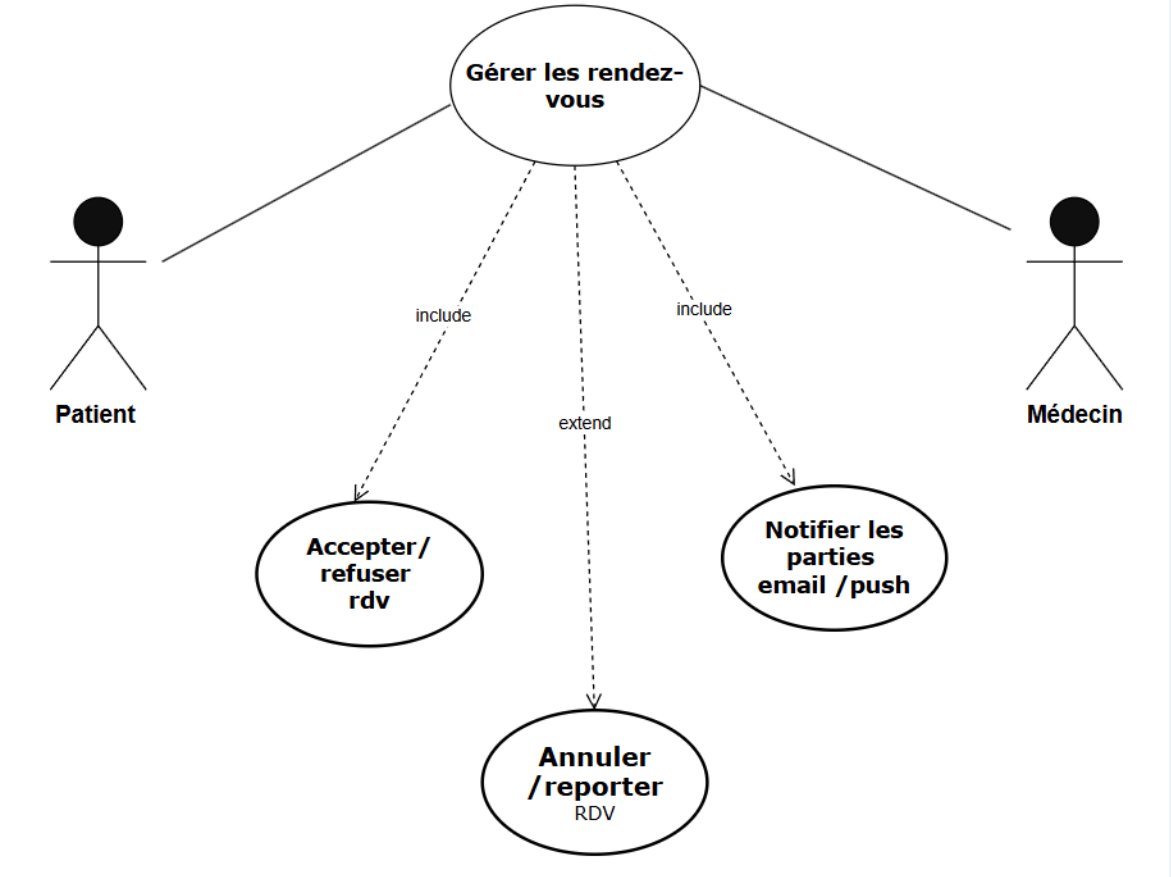


Figure 3.4 – Raffinement du cas « Gérer les rendez-vous »

Le Tableau 3.5 présente le raffinement du cas "Gérer les rendez-vous" :

Cas	Gérer les rendez-vous
Acteur	-Patient -Médecin
Condition préalable	L'utilisateur est authentifié.
Condition résultante	Rendez-vous modifié ou annulé avec succès.
Description de scénario	- Le médecin reçoit une notification instantanée lors d'une demande de RDV. - Le médecin accepte ou refuse la demande en 1 clic (statut VALIDÉ / REFUSÉ). - Le patient consulte ses rendez-vous depuis son espace personnel. - Le patient peut annuler ou reporter un RDV depuis son espace. - La plateforme notifie les deux parties de tout changement.
Scénarii	- Annulation tardive : la plateforme affiche un avertissement. - Refus médecin : notification envoyée au patient.

Tableau 3.5 – Raffinement du cas « Gérer les rendez-vous »

3.4 Conception de Release 1

3.4.1 Conception de Sprint 0

Conception du cas "S'inscrire"

Nous trouvons dans cette figure 3.5 le diagramme de classe associé au cas "S'inscrire patient" :

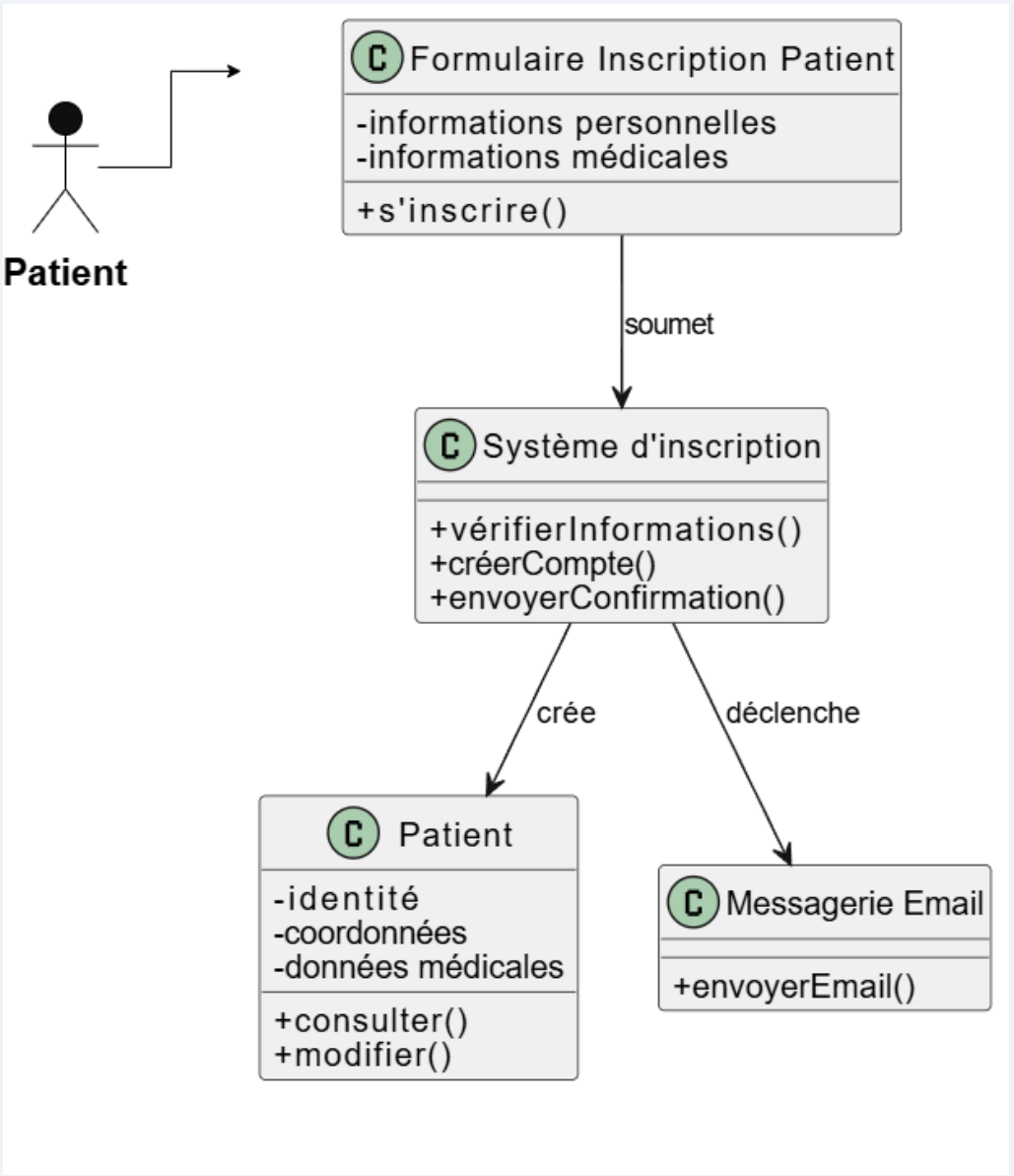


Figure 3.5 – Diagramme de classe conceptuel du cas « S'inscrire patient »

Dans la figure 3.6 nous pouvons observer le diagramme de classe du cas "S'inscrire Médecin" :

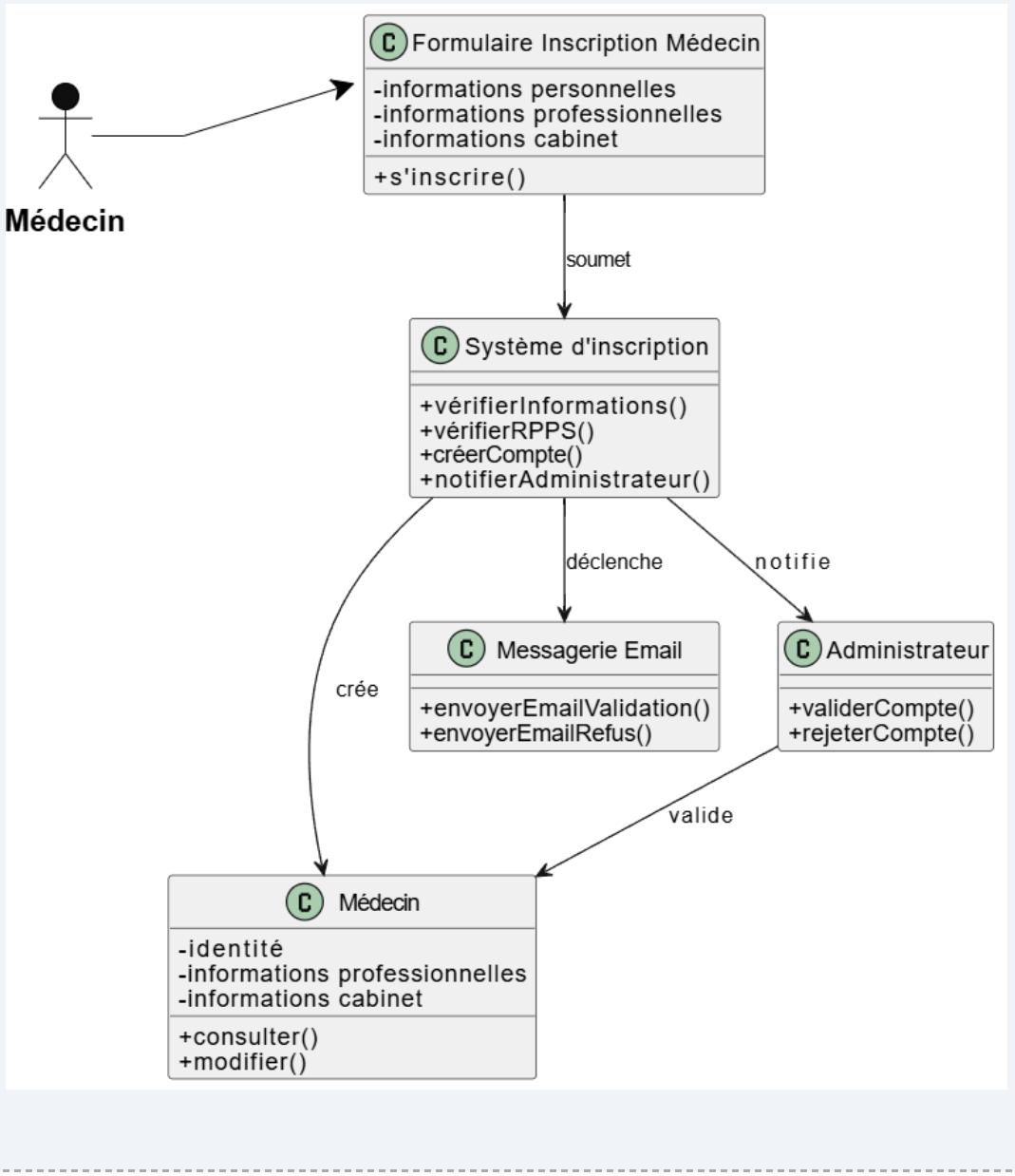


Figure 3.6 – Diagramme de classe conceptuel du cas d'utilisation « S'inscrire médecin »

Dans cette figure 3.7 on trouve le diagramme de séquence associé au cas "S'inscrire" :

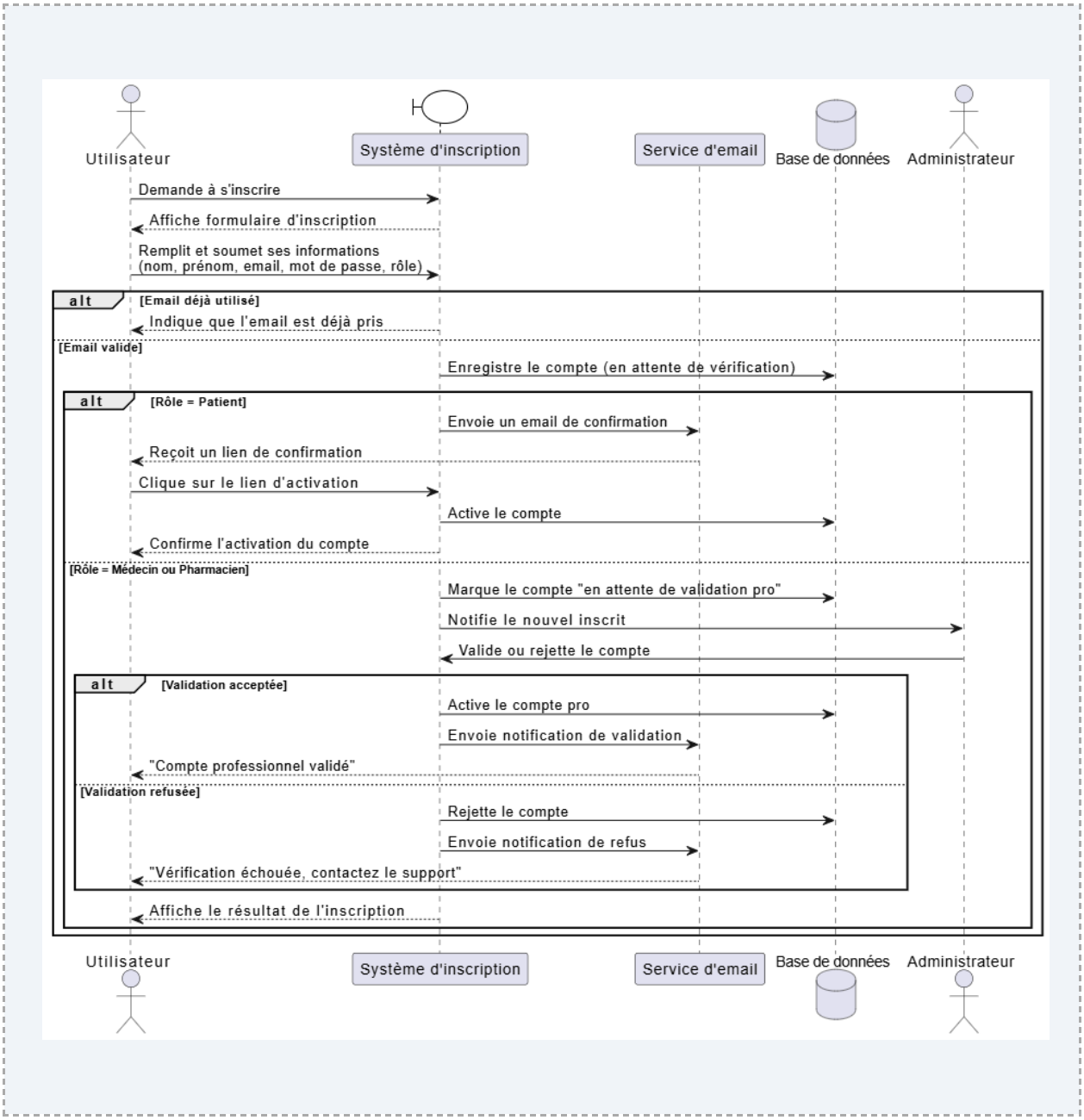


Figure 3.7 – Diagramme de séquence conceptuel du cas « S'inscrire »

Conception du cas "S'authentifier"

La figure 3.8 présente le diagramme de classe associé au cas "S'authentifier" :

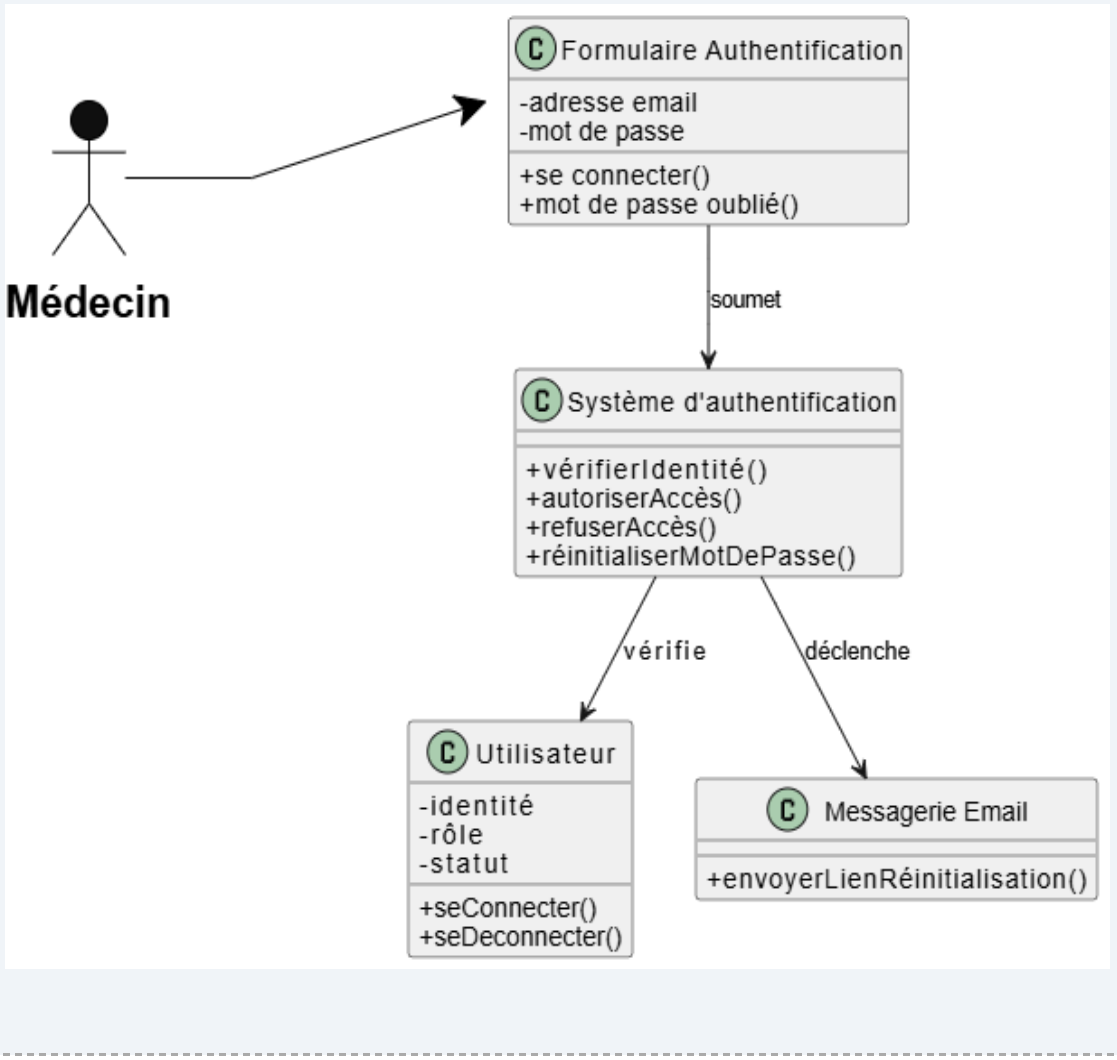


Figure 3.8 – Diagramme de classe Conceptuel du cas « S'authentifier »

La figure 3.9 présente le diagramme de séquence associé au cas "S'authentifier". Il illustre la séquence des messages échangés entre les différents objets du système :

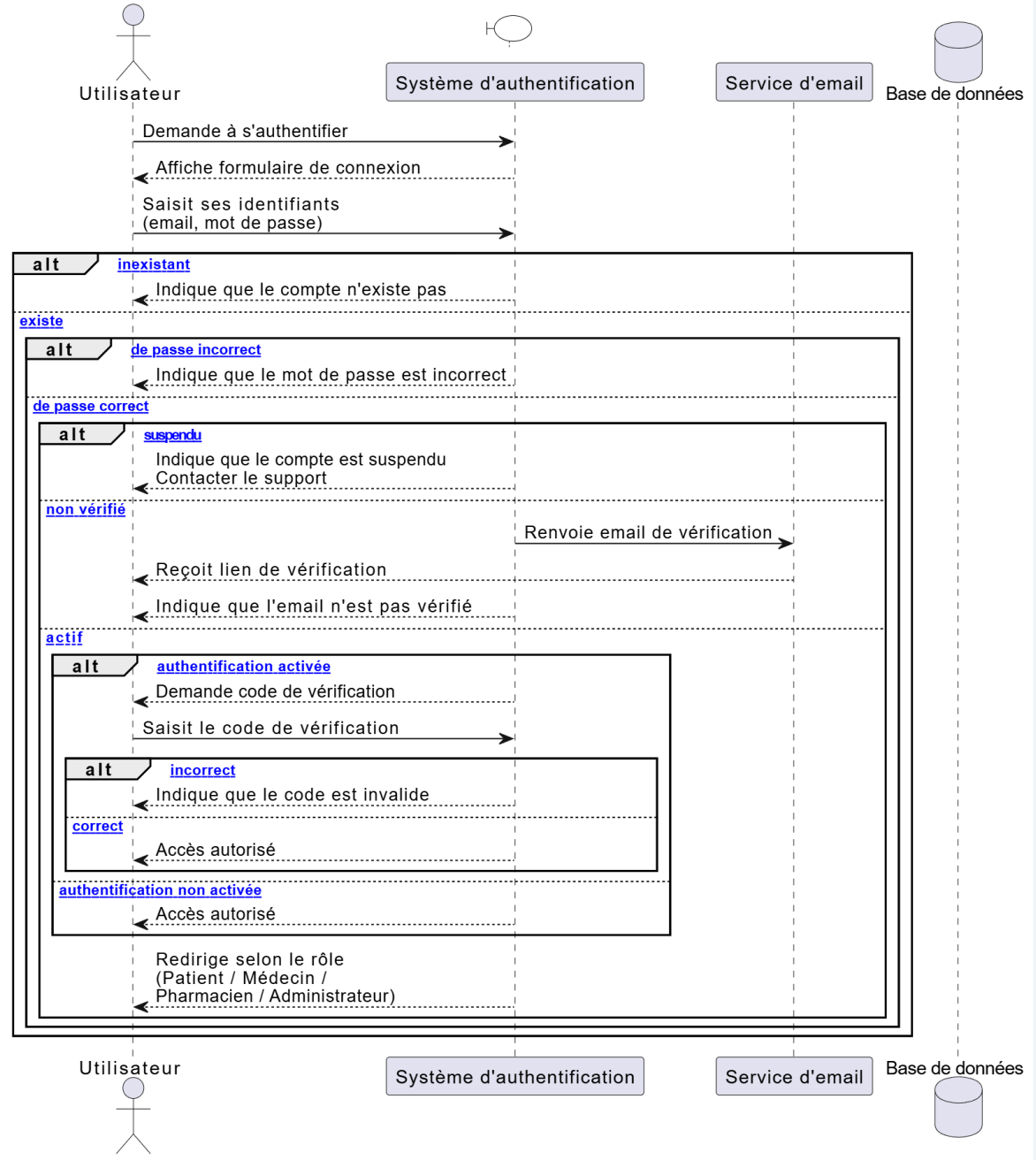


Figure 3.9 – Diagramme de séquence du cas « S'authentifier »

3.4.2 Conception de Sprint 1

Conception du cas "Prendre un rendez-vous"

Ce diagramme de classe 3.10 représente les différentes classes impliquées dans la prise de rendez-vous :

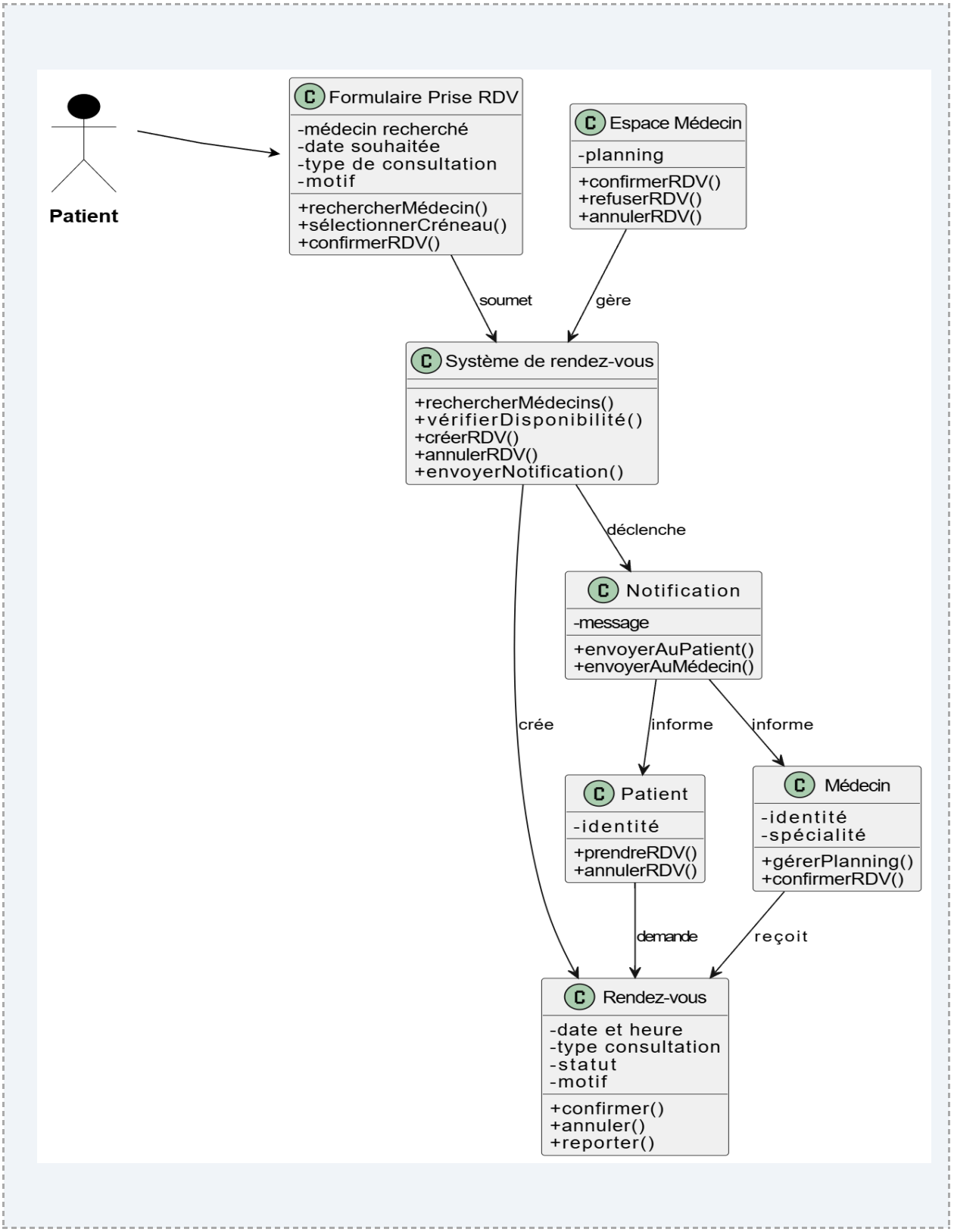


Figure 3.10 – Diagramme de classe Conceptuel du cas « Prendre un rendez-vous »

Ce diagramme de séquence 3.11 illustre la séquence des messages échangés entre les différents objets du cas "Prendre un rendez-vous" :

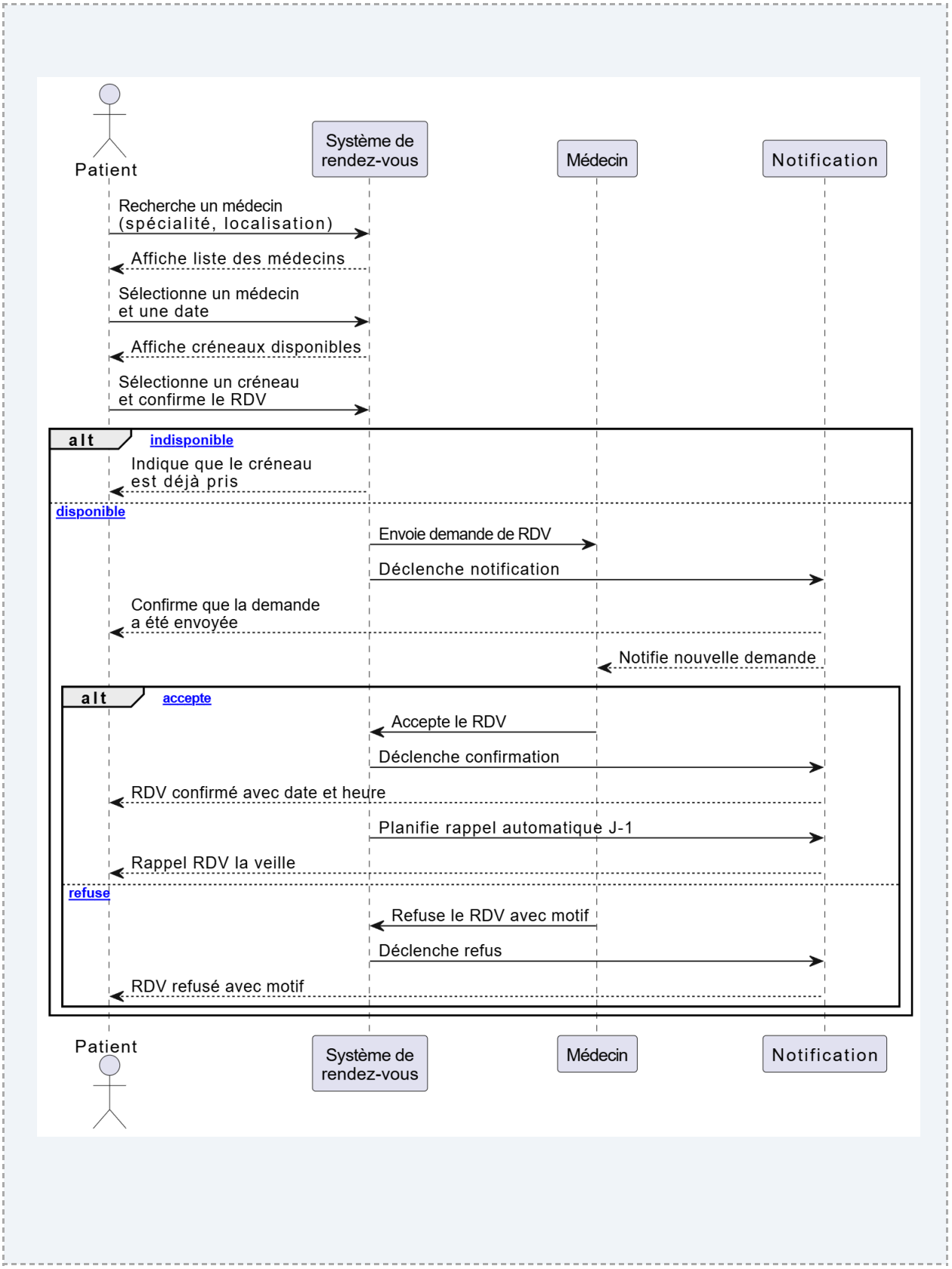


Figure 3.11 – Diagramme de séquence conceptuel du cas « Prendre un rendez-vous »

Conception du cas "Gérer les rendez-vous"

Ce diagramme de classe 3.12 illustre les différentes classes impliquées dans la gestion des rendez-vous :

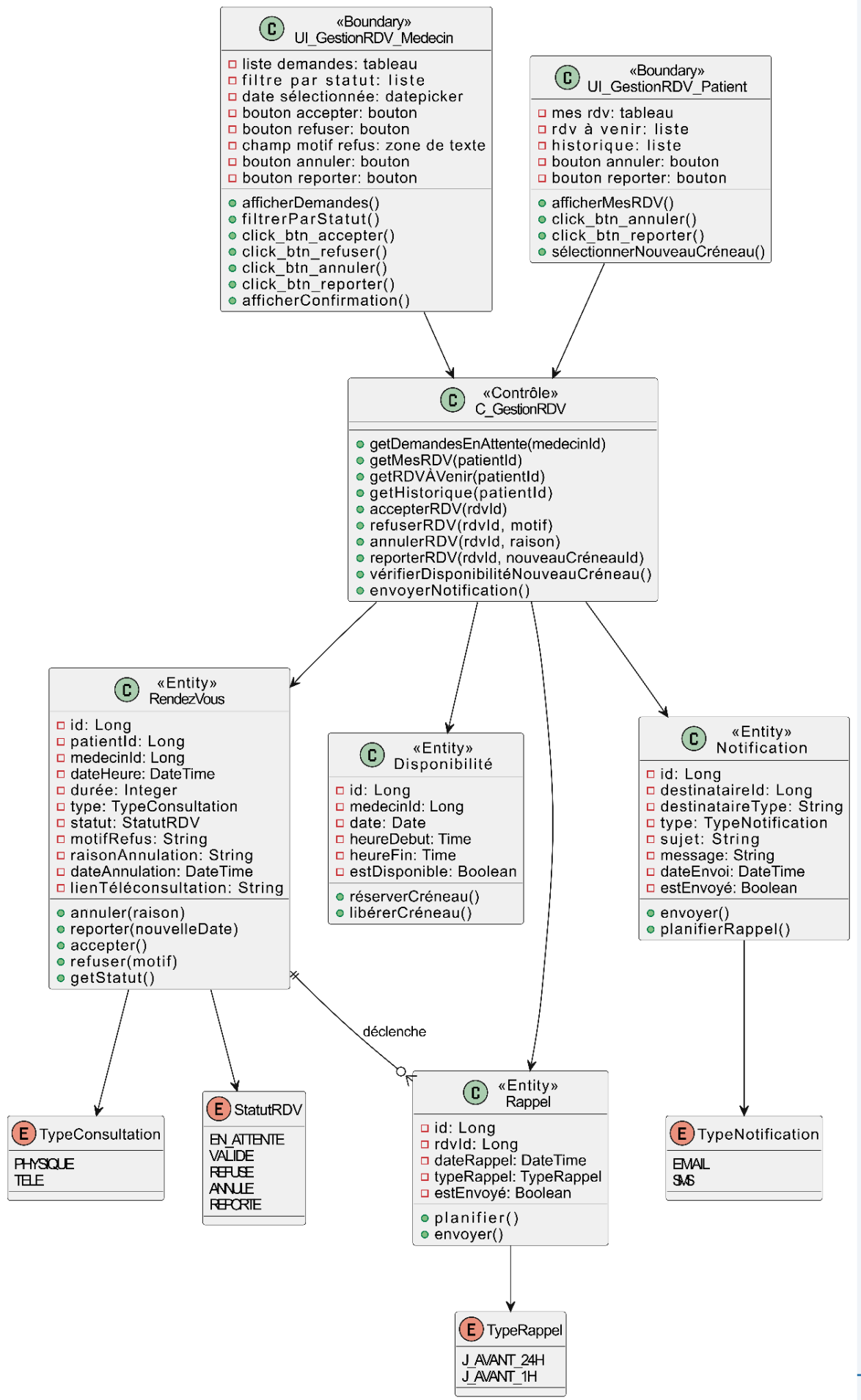

«Boundary»
UI_GestionRDV_Medecin
liste demandes: tableau
filtre par statut: liste
date sélectionnée: datepicker
bouton accepter: bouton
bouton refuser: bouton
champ motif refus: zone de texte
bouton annuler: bouton
bouton reporter: bouton
afficherDemandes()
filtrerParStatut()
click_btn_accepter()
click_btn_refuser()
click_btn_annuler()
click_btn_reporter()
afficherConfirmation()
«Boundary»
UI_GestionRDV_Patient
mes rdv: tableau
rdv à venir: liste
historique: liste
bouton annuler: bouton
bouton reporter: bouton
afficherMesRDV()
click_btn_annuler()
click_btn_reporter()
sélectionnerNouveauCréneau()
«Contrôle»
C_GestionRDV
getDemandesEnAttente(medecinId)
getMesRDV(patientId)
getRDVÀVenir(patientId)
getHistorique(patientId)
accepterRDV(rdvId)
refuserRDV(rdvId, motif)
annulerRDV(rdvId, raison)
reporterRDV(rdvId, nouveauCréneauId)
vérifierDisponibilitéNouveauCréneau()
envoyerNotification()
«Entity»
RendezVous
id: Long
patientId: Long
medecinId: Long
dateHeure: DateTime
durée: Integer
type: TypeConsultation
statut: StatutRDV
motifRefus: String
raisonAnnulation: String
dateAnnulation: DateTime
lienTéléconsultation: String
annuler(raison)
reporter(nouvelleDate)
accepter()
refuser(motif)
getStatut()
«Entity»
Disponibilité
id: Long
medecinId: Long
date: Date
heureDebut: Time
heureFin: Time
estDisponible: Boolean
réserverCréneau()
libérerCréneau()
«Entity»
Notification
id: Long
destinataireId: Long
destinataireType: String
type: TypeNotification
sujet: String
message: String
dateEnvoi: DateTime
estEnvoyé: Boolean
envoyer()
planifierRappel()
déclenche
«Entity»
Rappel
id: Long
rdvId: Long
dateRappel: DateTime
typeRappel: TypeRappel
estEnvoyé: Boolean
planifier()
envoyer()
TypeConsultation
PHYSIQUE
TELE
StatutRDV
EN_ATTENTE
VALIDE
REFUSE
ANNULE
REPORTE
TypeNotification
EMAIL
SMS
TypeRappel
J_AVANT_24H
J_AVANT_1H

Figure 3.12 – Diagramme de classe du cas « Gérer les rendez-vous »

Dans la figure 3.13, le diagramme de séquence du cas "Gérer les rendez-vous" illustre la séquence des interactions entre les différents acteurs et composants :

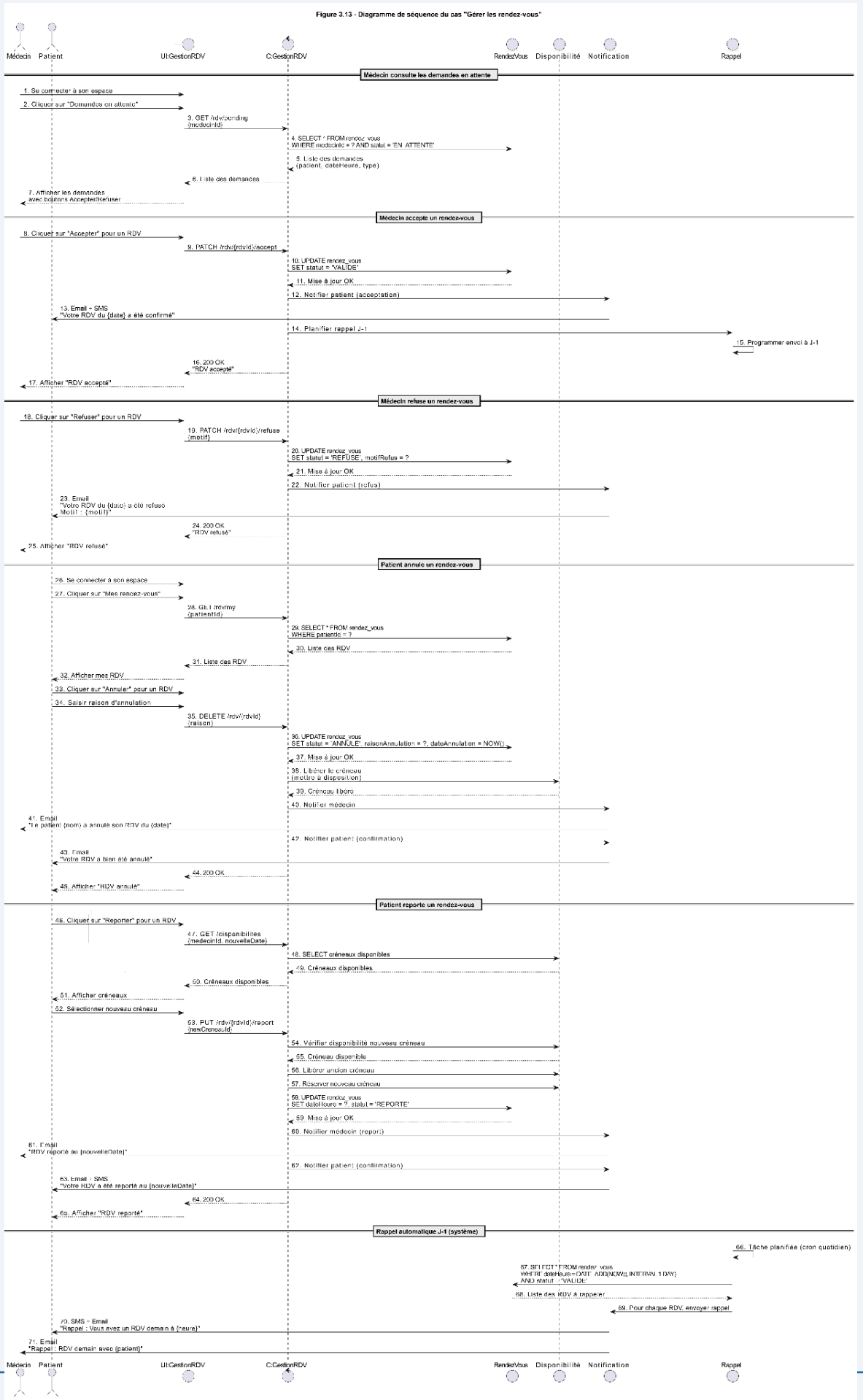

Figure 3.13 - Diagramme de séquence du cas "Gérer les rendez-vous"
Médecin
Patient
UI:GestionRDV
C:GestionRDV
RendezVous
Disponibilité
Notification
Rappel
Médecin consulte les demandes en attente
1. Se connecter à son espace
2. Cliquer sur "Demandes en attente"
3. GET /rdv/pending (medecinId)
4. SELECT * FROM rendez_vous WHERE medecinId = ? AND statut = 'EN_ATTENTE'
5. Liste des demandes (patient, date/heure, type)
6. Liste des demandes
7. Afficher les demandes avec boutons Accepter/Refuser
Médecin accepte un rendez-vous
8. Cliquer sur "Accepter" pour un RDV
9. PATCH /rdv/{rdvId}/accept
10. UPDATE rendez_vous SET statut = 'VALIDE'
11. Mise à jour OK
12. Notifier patient (acceptation)
13. Email + SMS "Votre RDV du {date} a été confirmé"
14. Planifier rappel J-1
15. Programmer envoi à J-1
16. 200 OK "RDV accepté"
17. Afficher "RDV accepté"
Médecin refuse un rendez-vous
18. Cliquer sur "Refuser" pour un RDV
19. PATCH /rdv/{rdvId}/refuse {motif}
20. UPDATE rendez_vous SET statut = 'REFUSE', motifRefus = ?
21. Mise à jour OK
22. Notifier patient (refus)
23. Email "Votre RDV du {date} a été refusé Motif : {motif}"
24. 200 OK "RDV refusé"
25. Afficher "RDV refusé"
Patient annule un rendez-vous
26. Se connecter à son espace
27. Cliquer sur "Mes rendez-vous"
28. GET /rdv/my {patientId}
29. SELECT * FROM rendez_vous WHERE patientId = ?
30. Liste des RDV
31. Liste des RDV
32. Afficher mes RDV
33. Cliquer sur "Annuler" pour un RDV
34. Saisir raison d'annulation
35. DELETE /rdv/{rdvId} {raison}
36. UPDATE rendez_vous SET statut = 'ANNULE', raisonAnnulation = ?, dateAnnulation = NOW()
37. Mise à jour OK
38. Libérer le créneau (motif: à disposition)
39. Créneau libéré
40. Notifier médecin
41. Email "Le patient {nom} a annulé son RDV du {date}"
42. Notifier patient (confirmation)
43. Email "Votre RDV a bien été annulé"
44. 200 OK
45. Afficher "RDV annulé"
Patient reporte un rendez-vous
46. Cliquer sur "Reporter" pour un RDV
47. GET /disponibilites {medecinId, nouvelleDate}
48. SELECT créneaux disponibles
49. Créneaux disponibles
50. Créneaux disponibles
51. Afficher créneaux
52. Sélectionner nouveau créneau
53. PUT /rdv/{rdvId}/report {newCreneauId}
54. Vérifier disponibilité nouveau créneau
55. Créneau disponible
56. Libérer ancien créneau
57. Réserver nouveau créneau
58. UPDATE rendez_vous SET dateHeure = ?, statut = 'REPORTE'
59. Mise à jour OK
60. Notifier médecin (report)
61. Email "RDV reporté au {nouvelleDate}"
62. Notifier patient (confirmation)
63. Email + SMS "Votre RDV a été reporté au {nouvelleDate}"
64. 200 OK
65. Afficher "RDV reporté"
Rappel automatique J-1 (système)
66. Tâche planifiée (cron quotidien)
67. SELECT * FROM rendez_vous WHERE dateHeure = DATE_ADD(NOW(), INTERVAL 1 DAY) AND statut = 'VALIDE'
68. Liste des RDV à rappeler
69. Pour chaque RDV, envoyer rappel
70. SMS + Email "Rappel : Vous avez un RDV demain à {heure}"
71. Email "Rappel : RDV demain avec {patient}"

Figure 3.13 – Diagramme de séquence du cas « Gérer les rendez-vous »

3.4.3 Diagramme de déploiement

La figure 3.14 présente le diagramme de déploiement. Il décrit l'architecture physique sur des composants matériels :

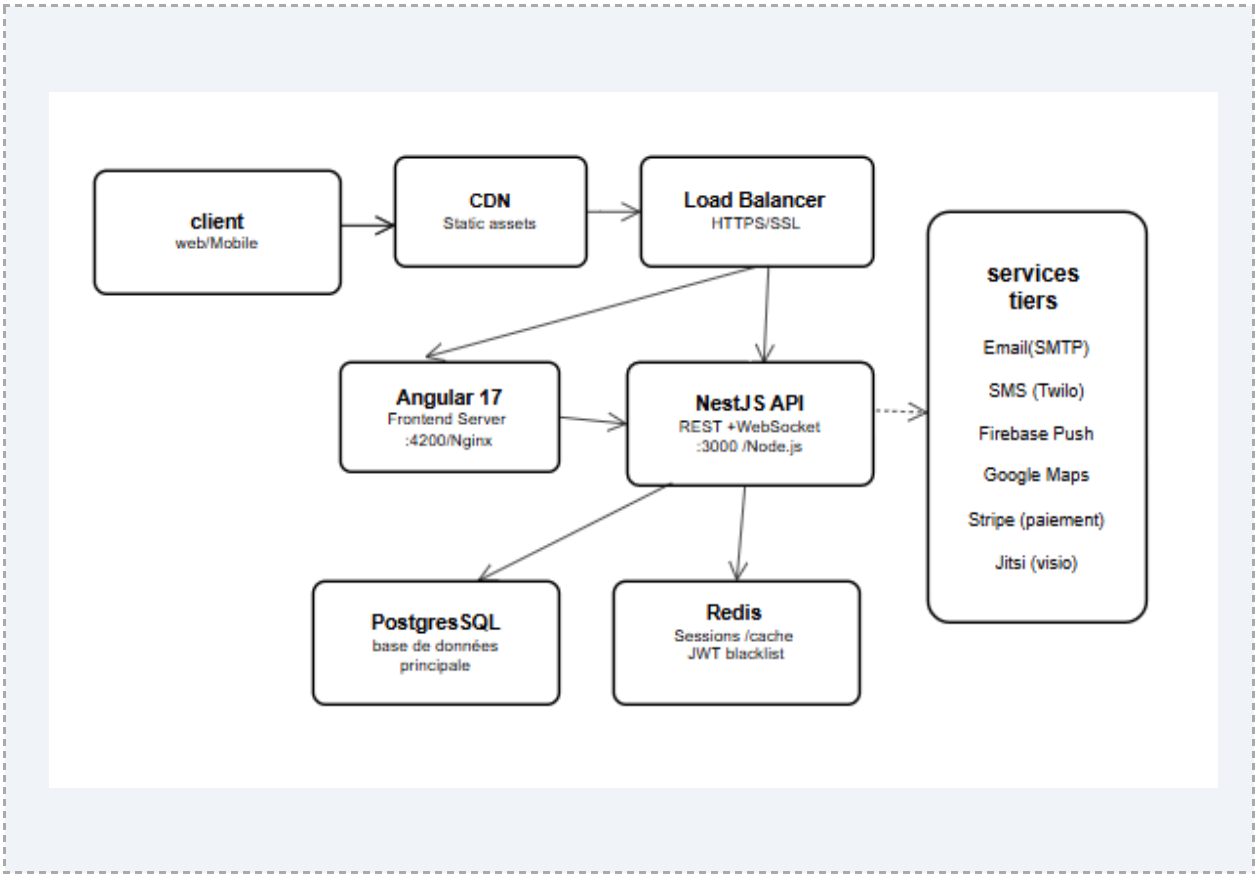


Figure 3.14 – Diagramme de déploiement